
PyGeoX

Release 0.2.0

Rafael Cabral

Aug 27, 2025

CONTENTS

1	Philosophy and Design	3
2	Getting Started with PyGeoX	5
2.1	Create a Scene	5
2.2	Add Objects	5
2.3	Define Relationships	5
2.4	Add Custom Constraints	6
3	Solving Geometric Problems	7
3.1	Visualize Results	7
4	Under Construction	9
4.1	Animation and Interactive Explorations	9
4.2	Automated Geometry Proofs	9
5	Examples	11
5.1	Diagram construction + calculation problem	11
5.2	Diagram construction + proof problem (Butterfly theorem)	12
5.3	Dynamic geometry with animation: two moving points	15
6	Main geometric scene	17
7	Add geometric objects	21
8	Add relationships	33
9	Add constraints	35
10	Solvers	37
11	Basic Objects (point, lines, circles, arcs)	41
12	Polygons	49
13	Animation (Under construction)	61
	Python Module Index	65

PyGeoX is an interactive Python library designed to create, manipulate, visualize, and solve geometric problems symbolically and numerically. Ideal for educational purposes, research, and interactive geometry explorations, PyGeoX simplifies geometric constructions and constraint solving with an intuitive, readable Python syntax.

PHILOSOPHY AND DESIGN

PyGeoX follows a object-oriented approach, where geometry becomes intuitive Python code. It supports creating geometric scenes, adding and relating geometric objects, defining precise constraints, and seamlessly solving complex geometry problems.

The philosophy behind PyGeoX revolves around simplicity, clarity, and flexibility:

- **Simple Syntax:** Clearly defined commands mimic geometric language.
 - **Automatic Constraint Management:** Constraints are implicitly added when relationships are defined.
 - **Rich Property Access:** Directly access properties like areas, lengths, midpoints, and more.
-

GETTING STARTED WITH PYGEOX

The main functionalities are described below. Please click **Examples** in the left panel to check several use cases.

2.1 Create a Scene

Start your geometry exploration by defining a scene:

```
from pygeox import GeoScene  
  
scene = GeoScene(10) # creates a scene with domain from -10 to 10
```

2.2 Add Objects

- PyGeoX contains **37 fundamental geometric objects**: Points, Lines, Segments, Rays, Circles, Arcs, Triangles, Quadrilaterals, Regular Polygons, and more.
- Please click **Add geometric objects** on the left panel to check all objects that can be added.
- PyGeoX makes creating geometric figures simple:

```
A, B, C = scene.add.points(['A', 'B', 'C'])  
triangle = scene.add.triangle(A, B, C)  
circle = scene.add.circle(center=A, radius=5)
```

- Each object has **2-15 detailed properties** such as area, perimeter, midpoint, centroid, circumcircle, incircle, altitudes, and medians.
- Example property access:

```
area = triangle.area  
length = line_AB.length  
mid_segment = triangle.midsegment
```

- Please click **Basic Objects and Polygons** on the left panel to see all object properties.

2.3 Define Relationships

- PyGeoX contains **27 geometric relationships**: parallel, tangent, similarity, inside/outside containment, etc.
- Please click **Add relationships** on the left panel to check all relationships that can be added.

- Relationships automatically introduce constraints to scene:

```
# Make line AB parallel to line CD
scene.relate.parallel(line_AB, line_CD)

# Point P lies on circle
scene.relate.point_lies_on(P, circle)
```

- This implicitly adds constraints like:

```
# Example internal constraint:
# (line_AB.direction_vector × line_CD.direction_vector) = 0
```

2.4 Add Custom Constraints

- Please click Add constraints on the left panel to check all constraints that can be added.
- Define explicit mathematical constraints:

```
import sympy
scene.constraint.eq(triangle.area, sympy.cos(circle.perimeter) / circle.area)
scene.constraint.gt(line_AB.length, circle.radius)
```

SOLVING GEOMETRIC PROBLEMS

- PyGeoX provides both analytical (symbolic) and numerical solvers.

```
scene.solver.analytical()  
scene.solver.numerical(method="basinhopping")
```

- The numerical solver avoid degenerate solutions (e.g., points too close, radius too small) by adding extra constraints.
- Please click Solvers on the left panel to see all available options.

3.1 Visualize Results

Easily visualize solutions with built-in plotting:

```
scene.plot()
```

UNDER CONSTRUCTION

4.1 Animation and Interactive Explorations

Future PyGeoX versions will support animations and dynamic geometry, such as moving points:

```
anim_lambda = scene.animate.add_parameter("lambda")
scene.animate.run(frames=50)
scene.animate.gif("animation.gif")
```

4.2 Automated Geometry Proofs

PyGeoX is developing an integrated proving system, enabling automated geometric proofs:

```
with scene.proving() as prover:
    scene.relate.collinear(A, B, C)

scene.prove()
```


EXAMPLES

5.1 Diagram construction + calculation problem

Problem: In a regular hexagon ABCDEF, a circle O passing through points E and F is tangent to sides AB and CD at points G and H, respectively, and intersects side DE at point M. Lines GM and FH intersect at point N, the measure of angle GNF is ____°.

```
from pygeox import GeoScene, LineSegment

scene = GeoScene(10)

A, B, C, D, E, F, G, H, M, N, O = scene.add.points(["A", "B", "C", "D", "E", "F",
↪ "G", "H", "M", "N", "O"])

# In a regular hexagon ABCDEF
hexagon = scene.add.regular_hexagon(A, B, C, D, E, F)

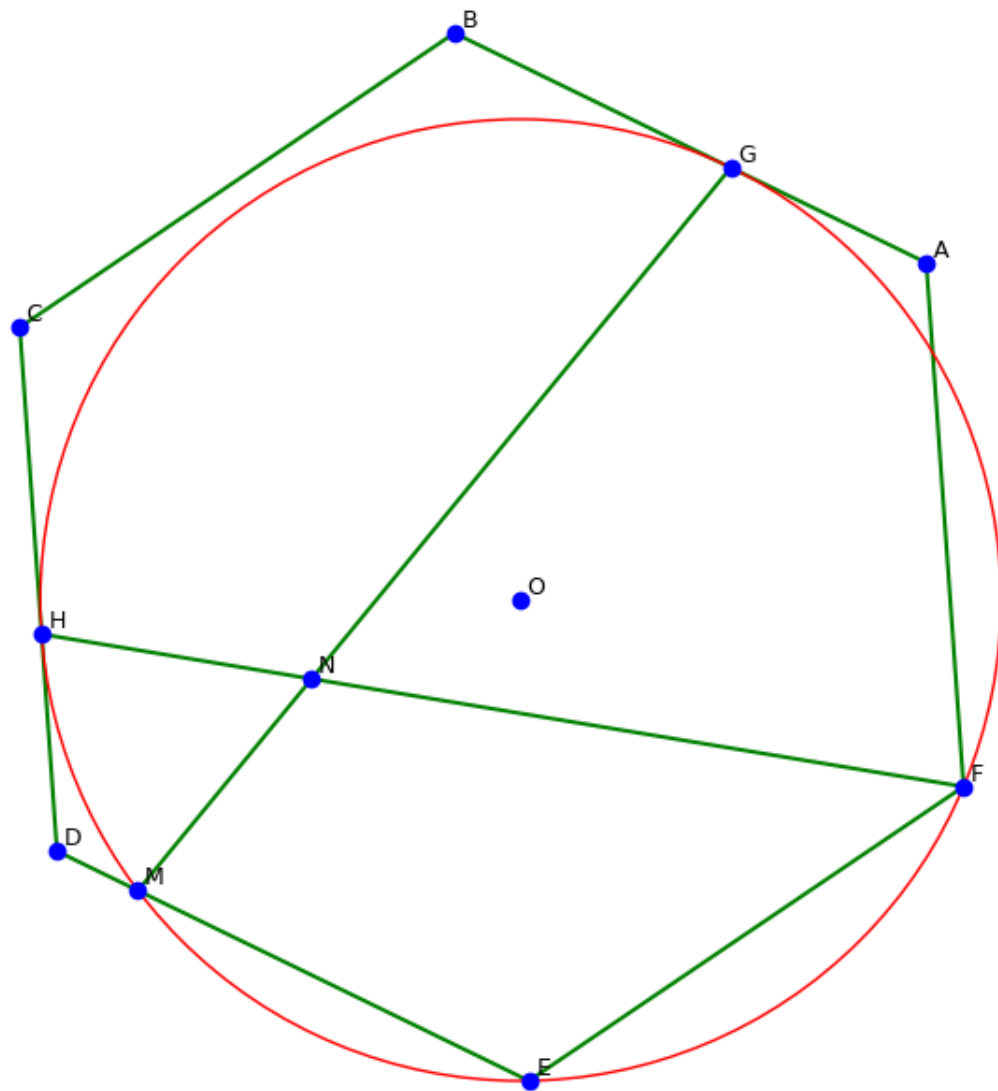
# a circle O passing through points E and F
circle = scene.add.circle(O)
scene.relate.points_lie_on([E, F], circle)

# tangent to sides AB and CD at points G and H
scene.relate.tangent_to_circle(LineSegment(A,B), circle, G)
scene.relate.tangent_to_circle(LineSegment(C,D), circle, H)

## intersects side DE at point M
scene.relate.line_intersects_circle_at(LineSegment(D,E), circle, M)

## Lines GM and FH intersect at point N
Line_GM = scene.add.line_segment(G, M)
Line_FH = scene.add.line_segment(F, H)
scene.relate.lines_intersect_at(Line_GM, Line_FH, N)

scene.solver.numerical()
scene.plot()
```



The function `scene.solver.numerical()` finds one of possibly many valid configurations for the diagram. After, we can call any object property to find its numerical value.

```
# the measure of angle GNF is ____°
scene.angle(G,N,F).evalf(4)
#60
```

5.2 Diagram construction + proof problem (Butterfly theorem)

Problem: Let M be the midpoint of a chord PQ of a circle, through which two other chords AB and CD are drawn; AD and BC intersect chord PQ at X and Y correspondingly. Prove that M is the midpoint of XY .


```

from pygeoX import GeoScene, LineSegment

scene = GeoScene(5)

M, P, Q, A, B, C, D, X, Y, O = scene.add.points(["M", "P", "Q", "A", "B", "C", "D", "X",
↪ "Y", "O"])

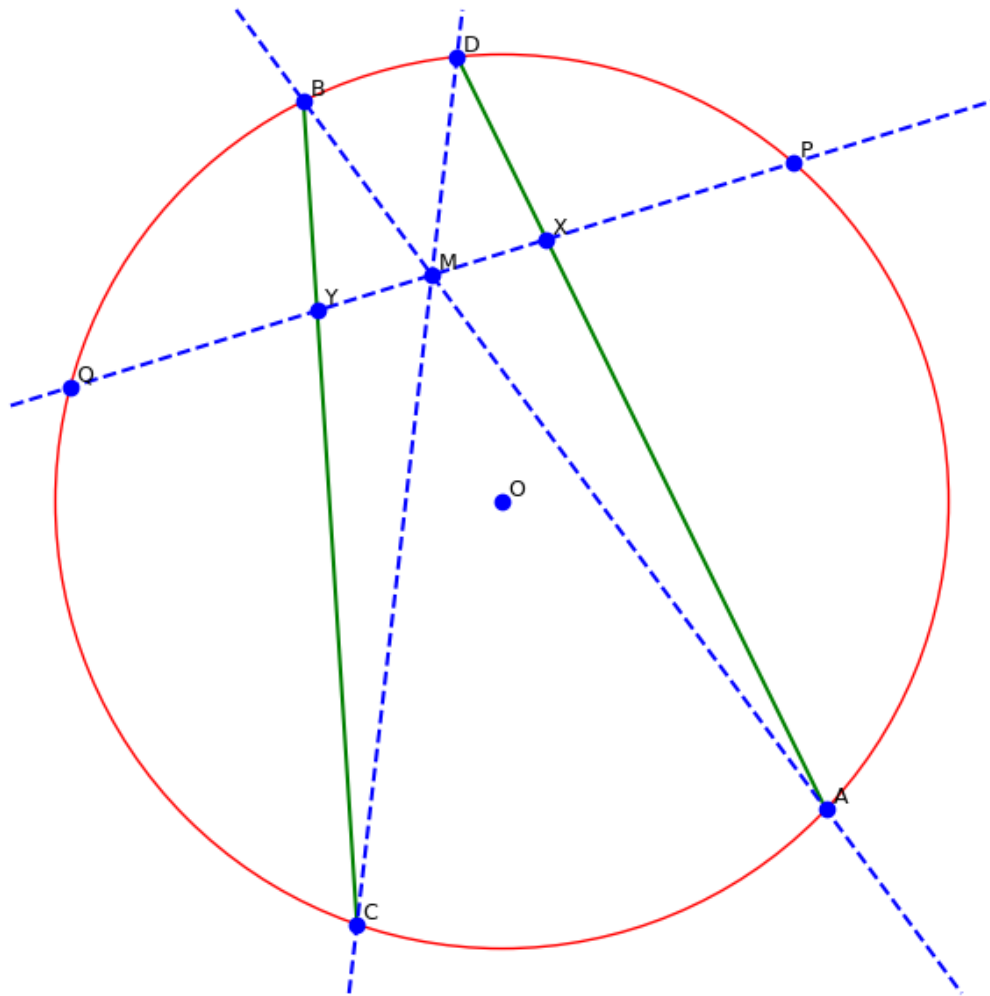
# Let M be the midpoint of a chord PQ of a circle, through which two other ↪
↪ chords AB and CD are drawn
circle = scene.add.circle(O)
l_AB = scene.add.chord(circle, A, B)
l_CD = scene.add.chord(circle, C, D)
l_PQ = scene.add.chord(circle, P, Q)
scene.relate.is_midpoint(M, l_PQ)
scene.relate.point_lies_on(M, l_CD)
scene.relate.point_lies_on(M, l_AB)

# AD and BC intersect chord PQ at X and Y correspondingly
l_AD = scene.add.line_segment(A, D)
l_BC = scene.add.line_segment(B, C)
scene.relate.lines_intersect_at(l_AD, l_PQ, X)
scene.relate.lines_intersect_at(l_BC, l_PQ, Y)

# Then M is the midpoint of XY.
with scene.proving():
    scene.relate.is_midpoint(M, LineSegment(X, Y))

scene.solver.numerical(distance_penalty=0.01)
scene.plot()

```



Our built-in prover can prove “M is the midpoint of XY” based on the stated conditions.

```
scene.prove()
#Conclusion: 2*x_M - x_X - x_Y = 0
#Final Remainder: 0
#Verdict: PROVEN TRUE. The conclusion is a consequence of the hypotheses.

#Conclusion: 2*y_M - y_X - y_Y = 0
#Final Remainder: 0
#Verdict: PROVEN TRUE. The conclusion is a consequence of the hypotheses.
```

5.3 Dynamic geometry with animation: two moving points

Problem: In the rhombus ABCD, draw line segment DE perpendicular to CD, where E is the point of intersection with diagonal AC. Connect BE. Point P is a moving point on segment BE, and construct the symmetry point P' of point P with respect to line DE. Point Q is a moving point on diagonal AC, connecting PQ and DQ. If AE equals 14 and CE equals 18, then the maximum value of $DQ - P'Q$ is ____.

```
from pygeox import GeoScene, LineSegment

# Create a new scene
scene = GeoScene(5)

# add objects
A, B, C, D, E, P, Q = scene.add.points(["A", "B", "C", "D", "E", "P", "Q"])

# In the rhombus ABCD
scene.add.rhombus(A, B, C, D)

# draw line segment DE perpendicular to CD
line_DE = scene.add.line_segment(D, E)
line_CD = scene.add.line_segment(C, D)
scene.relate.perpendicular(line_DE, line_CD)

#where E is the point of intersection with diagonal AC
line_AC = scene.add.line_segment(A, C)
scene.relate.point_lies_on(E, line_AC)

#Connect BE
line_BE = scene.add.line_segment(B, E)

#Point P is a moving point on segment BE
scene.animate.point_sweep_line(point = P, start_point = B, end_point = E)

#construct the symmetry point P' of point P with respect to line DE
Ps = scene.add.point("P'", copy = P.mirror_across_line(line_DE))

#Point Q is a moving point on diagonal AC, connecting PQ and DQ
scene.animate.point_sweep_line(point = Q, start_point = A, end_point = C)
line_PQ = scene.add.line_segment(P, Q)
line_DQ = scene.add.line_segment(D, Q)

# If AE equals 14 and CE equals 18
scene.constraint.eq(LineSegment(A, E).length, 1.4) # use small numbers 14 -> 1.4
scene.constraint.eq(LineSegment(C, E).length, 1.8) # use small numbers 18 -> 1.8

#run animation
scene.animate.run(moving_points= [P, Ps, Q], opt_method = "dual_annealing",
frames = 32)
#when default optimizer fails, use basinhopping -> slower but it works better
scene.animate.gif(total_time=10)
```


MAIN GEOMETRIC SCENE

GeoScene module: Core framework for geometric scene management, constraint handling, and problem solving.

This module defines the GeoScene class, which serves as the central manager for geometric objects (points, lines, circles, polygons, arcs) and constraints. It provides a unified API to add, retrieve, and manipulate geometric elements, define symbolic constraints between them, and invoke solvers to find solutions to geometric problems.

Key features include: - Automatic management and storage of geometric objects. - Symbolic constraint definition with human-readable descriptions. - Integration of namespaces for relationships, constraint addition, solving,

animation, and geometric proofs.

- Sketching capability by auto-creating points when referenced by name.
- Utilities for calculating angles, plotting scenes, and proof management.

The GeoScene class acts as the main entry point for users to programmatically create, analyze, and visualize geometry problems and their solutions.

class pygeox.geoscene.**GeoScene**(*domain=None*)

Bases: object

Manages all geometric objects and constraints, and solves for unknown variables. This class is the main entry point for defining, manipulating, and solving geometric problems. It provides sketching capabilities by automatically creating points when referenced by name.

Parameters

domain (*Any* / *None*)

class ProvingContext(*scene_instance*)

Bases: object

Initialize proving context manager.

Parameters

scene_instance – GeoScene instance.

add_constraint(*equation, description*)

Add a symbolic equation constraint with a description.

If in proving mode, stores constraint for proof; else adds to main constraints.

Parameters

- **equation** (*Basic*) – Sympy symbolic equation.
- **description** (*str*) – Human-readable description.

Return type

None

add_mapping_constraint(*equation, description*)

Add a mapping constraint equation used for parameter mapping.

Parameters

- **equation** (*Basic*) – Sympy symbolic equation.
- **description** (*str*) – Human-readable description.

Return type

None

add_object(*obj_type, name, obj*)

Add a geometric object to the scene under the given type and name.

Parameters

- **obj_type** (*str*) – Type of the object (e.g., 'Point', 'Circle').
- **name** (*str*) – Name of the object.
- **obj** (*Any*) – The geometric object instance.

Return type

None

add_parameter(*symbol, type='x', full=True, origin='Problem', bound=None*)

Add a symbolic parameter with metadata to the scene.

Parameters

- **symbol** (*Symbol*) – The sympy.Symbol representing the parameter.
- **type** (*str*) – Parameter type (default "x").
- **full** (*bool*) – Whether parameter is fully active.
- **origin** (*str*) – Origin description.
- **bound** – Optional numeric bounds [min, max].

Returns

The symbol added.

angle(*point1, vertex, point2, signed=False*)

Calculate the angle (degrees between 0 and 180) at vertex between point1 and point2.

Parameters

- **point1** (*Point*) – First point defining the angle.
- **vertex** (*Point*) – Vertex point of the angle.
- **point2** (*Point*) – Second point defining the angle.
- **signed** (*bool*) – If True, returns signed angle (CCW positive).

Returns

Angle in degrees.

Return type

int | float | *Expr*

get_all_objects(*obj_type=None*)

Retrieve all objects of a given type or all objects if no type is specified.

Parameters

obj_type (*str* | *None*) – Optional type of objects to retrieve.

Returns

Dictionary of object names to objects.

Return type

Dict[*str*, *Any*]

get_object(*obj_type, name*)

Retrieve a specific object by type and name.

Parameters

- **obj_type** (*str*) – Type of the object.
- **name** (*str*) – Name of the object.

Returns

The geometric object or None if not found.

Return type

Any | None

get_points()

Get dictionary of all points in the scene with their (x, y) coordinates.

Returns

Dict mapping point names to (x, y) coordinate tuples.

Return type

dict

plot(*show_names=True, show_axis_grid=False, xlim=None, ylim=None, return_fig=False*)

Plot all geometric objects in the scene.

Parameters

- **show_names** (*bool*) – Show object names on plot.
- **show_axis_grid** (*bool*) – Show axis grid.
- **xlim** (*tuple[float, float] | None*) – X-axis limits.
- **ylim** (*tuple[float, float] | None*) – Y-axis limits.
- **return_fig** (*bool*) – If True, return matplotlib figure object.

Returns

Matplotlib figure if *return_fig* is True; otherwise None.

prove()

Run the prover function on the accumulated proof statements and constraints. Updates the *proof_statements* with prover results.

proving()

Create a context manager for defining proof constraints.

Returns

ProvingContext instance.

ADD GEOMETRIC OBJECTS

Factory namespace for creating and adding geometric objects to a scene.

This module defines the `AddNamespace` class, which provides convenient constructors to create geometric objects such as points, lines, arcs, polygons, and various special polygons (triangles, quadrilaterals, regular polygons, etc.).

Each method creates the object, registers it in the scene, and adds necessary constraints automatically (e.g., equal radii for arcs, collinearity for semicircles).

The namespace facilitates easy and consistent object creation for geometric problem modeling, with support for copying existing objects and symbolic coordinate handling.

class `pygeox.add_objects.AddNamespace(scene)`

Bases: `object`

Factory namespace for adding geometric objects to a scene.

This class exposes convenience constructors that create objects, register them in the scene, and add any required constraints.

acute_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add an acute triangle or return a copy.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **name** (*str* | *None*) – Optional name.
- **copy** (*AcuteTriangle* | *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

`AcuteTriangle`

Notes

All internal angles are constrained $< 90^\circ$.

angle(*A, vertex, B, plot_sign=True, plot_text=False, name=None*)

Add an angle object defined by three points: A, vertex, and B.

The angle is formed at the *vertex* point between points A and B.

Parameters

- **A** (*Point*) – One point defining the angle.
- **vertex** (*Point*) – The vertex point where the angle is measured.
- **B** (*Point*) – The other point defining the angle.

- **name** (*Optional[str], optional*) – Name for the angle object.
- **plot_sign** (*bool | None*)
- **plot_text** (*bool | None*)

Returns

The measure of the angle at *vertex* between points *A* and *B*. This is typically a symbolic expression (e.g., a SymPy Expr) if coordinates are symbolic or unknown.

chord(*circle, A, B, name=None*)

Add a chord between two points on a circle.

Points *A* and *B* are constrained to lie on *circle*.

Parameters

- **circle** (*Circle*) – The circle on which the points lie.
- **A** (*Point*) – First point on the circle.
- **B** (*Point*) – Second point on the circle.
- **name** (*str | None*) – Optional name. Defaults to `chord_A_B_on_<circle>`.

Returns

The created chord, registered in the scene.

Return type

LineSegment

circle(*center=None, radius=None, name=None, copy=None*)

Add a circle or copy an existing one.

If not copying and *radius* is *None*, a `sympy.Symbol` is created for the radius using the scene's radius domain.

Parameters

- **center** (*Point | None*) – Center point of the circle.
- **radius** (*int | float | Expr | None*) – Radius value or SymPy expression. If *None*, a symbol is created.
- **name** (*str | None*) – Optional name for the circle.
- **copy** (*Circle | None*) – Existing circle to duplicate.

Returns

The created circle, registered in the scene.

Return type

Circle

Raises

ValueError – If not copying and *center* is not provided.

equilateral_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add an equilateral triangle or return a copy.

Parameters

- **p1** (*Point | None*) – Vertex A.
- **p2** (*Point | None*) – Vertex B.
- **p3** (*Point | None*) – Vertex C.
- **name** (*str | None*) – Optional name.
- **copy** (*EquilateralTriangle | None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

EquilateralTriangle

Notes

Three distinct points are required; all sides are constrained equal.

isosceles_trapezoid(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add an isosceles trapezoid or return a copy.

Constrains AB to be parallel to CD and BC = DA. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*IsoscelesTrapezoid* | *None*) – Existing trapezoid to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

IsoscelesTrapezoid

isosceles_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add an isosceles triangle or return a copy.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B (apex for equal sides AB and BC).
- **p3** (*Point* | *None*) – Vertex C.
- **name** (*str* | *None*) – Optional name.
- **copy** (*IsoscelesTriangle* | *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

IsoscelesTriangle

Notes

Sides AB and BC are constrained equal (order matters).

kite(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a kite or return a copy.

Constrains AB = BC and CD = DA. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Kite* | *None*) – Existing kite to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Kite

line(*point1=None, point2=None, name=None, copy=None*)

Add a line or copy an existing one.

If *copy* is provided and *point1/point2* are given, their coordinates are overwritten to match the copy. A default name is inferred if none is given.

Parameters

- **point1** (*Point* | *None*) – First point on the line.
- **point2** (*Point* | *None*) – Second point on the line.
- **name** (*str* | *None*) – Optional name for the line.
- **copy** (*Line* | *None*) – Existing line to duplicate.

Returns

The created line, registered in the scene.

Return type

Line

Raises

ValueError – If not copying and fewer than two points are provided.

line_segment(*point1=None, point2=None, name=None, copy=None*)

Add a line segment or copy an existing one.

Parameters

- **point1** (*Point* | *None*) – First endpoint.
- **point2** (*Point* | *None*) – Second endpoint.
- **name** (*str* | *None*) – Optional name for the segment.
- **copy** (*LineSegment* | *None*) – Existing segment to duplicate.

Returns

The created segment, registered in the scene.

Return type

LineSegment

Raises

ValueError – If not copying and fewer than two points are provided.

major_arc(*center, start_point, end_point, name=None, on_circle=None*)

Create a `MajorArc` and register it in the scene.

Adds equal-radius constraints and optionally enforces that the arc lies on a given circle (same center and radius).

Parameters

- **center** (*Point*) – Center of the circle.
- **start_point** (*Point*) – Start point of the arc.
- **end_point** (*Point*) – End point of the arc.
- **name** (*str* | *None*) – Optional name. If *None*, one is generated.
- **on_circle** (*Circle* | *None*) – If provided, enforces arc radius equals the circle's radius.

Returns

The created arc.

Return type

MajorArc

minor_arc(*center, start_point, end_point, name=None, on_circle=None*)

Create a `MinorArc` and register it in the scene.

Adds equal-radius constraints and optionally enforces that the arc lies on a given circle (same center and radius).

Parameters

- **center** (*Point*) – Center of the circle.
- **start_point** (*Point*) – Start point of the arc.

- **end_point** (*Point*) – End point of the arc.
- **name** (*str* | *None*) – Optional name. If *None*, one is generated.
- **on_circle** (*Circle* | *None*) – If provided, enforces arc radius equals the circle's radius.

Returns

The created arc.

Return type

MinorArc

obtuse_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add an obtuse triangle or return a copy.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **name** (*str* | *None*) – Optional name.
- **copy** (*ObtuseTriangle* | *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

ObtuseTriangle

Notes

At least one internal angle is constrained $> 90^\circ$.

parallelogram(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a parallelogram or return a copy.

Opposite sides are constrained parallel. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If *True*, enforces convexity and simplicity.
- **copy** (*Parallelogram* | *None*) – Existing parallelogram to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Parallelogram

point(*name=None, x=None, y=None, copy=None*)

Add a point or copy an existing one.

If *copy* is provided, its coordinates and name are used unless overridden. If *x* or *y* is *None* (and not copying), a *sympy.Symbol* is created for that coordinate, using any domain set on the scene.

Parameters

- **name** (*str*) – Optional name for the point.
- **x** (*int* | *float* | *Expr* | *None*) – X coordinate (number or SymPy expression). If *None*, a symbol is created.
- **y** (*int* | *float* | *Expr* | *None*) – Y coordinate (number or SymPy expression). If *None*, a symbol is created.
- **copy** (*Point* | *None*) – Existing point to duplicate.

Returns

The created point, registered in the scene.

Return type

Point

points(*names*)

Add multiple points by name.

Parameters

names (*list[str]*) – List of point names.

Returns

The created points.

Return type

list[Point]

polygon(**points, name=None, copy=None*)

Add a general polygon or return a copy.

Assumptions & constraints (when creating): at least three points, given in order. If *copy* is provided and the number of points matches, provided points are overwritten to match.

Parameters

- ***points** (*Point*) – Vertices in order.
- **name** (*str | None*) – Optional name.
- **copy** (*Polygon | None*) – Existing polygon to reuse.

Returns

The polygon instance (copy if provided).

Return type

Polygon

quadrilateral(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a quadrilateral or return a copy.

Parameters

- **p1** (*Point | None*) – Vertex A.
- **p2** (*Point | None*) – Vertex B.
- **p3** (*Point | None*) – Vertex C.
- **p4** (*Point | None*) – Vertex D.
- **name** (*str | None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Quadrilateral | None*) – Existing quadrilateral to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Quadrilateral

ray(*start_point=None, direction_point=None, name=None, copy=None*)

Add a ray or copy an existing one.

Parameters

- **start_point** (*Point | None*) – Ray origin.
- **direction_point** (*Point | None*) – Second point indicating direction.
- **name** (*str | None*) – Optional name for the ray.
- **copy** (*Ray | None*) – Existing ray to duplicate.

Returns

The created ray, registered in the scene.

Return type

Ray

Raises

ValueError – If not copying and a start/direction point is missing.

rectangle(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a rectangle or return a copy.

All angles are constrained 90°; opposite sides parallel. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Rectangle* | *None*) – Existing rectangle to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Rectangle

regular_heptagon(**points, name=None, copy=None*)

Add a regular heptagon or return a copy.

Assumptions & constraints: exactly seven ordered points; all sides and angles equal.

Parameters

- ***points** (*Point*) – Seven vertices in order.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RegularHeptagon* | *None*) – Existing heptagon to reuse.

Returns

The polygon instance (copy if provided).

Return type

RegularHeptagon

regular_hexagon(**points, name=None, copy=None*)

Add a regular hexagon or return a copy.

Assumptions & constraints: exactly six ordered points; all sides and angles equal; inscribed in a circle.

Parameters

- ***points** (*Point*) – Six vertices in order.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RegularHexagon* | *None*) – Existing hexagon to reuse.

Returns

The polygon instance (copy if provided).

Return type

RegularHexagon

regular_octagon(**points, name=None, copy=None*)

Add a regular octagon or return a copy.

Assumptions & constraints: exactly eight ordered points; all sides and angles equal.

Parameters

- ***points** (*Point*) – Eight vertices in order.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RegularOctagon* | *None*) – Existing octagon to reuse.

Returns

The polygon instance (copy if provided).

Return type

RegularOctagon

regular_pentagon(*points, name=None, copy=None)

Add a regular pentagon or return a copy.

Assumptions & constraints: exactly five ordered points; all sides and angles equal; inscribed in a circle.

Parameters

- ***points** (*Point*) – Five vertices in order.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RegularPentagon* | *None*) – Existing pentagon to reuse.

Returns

The polygon instance (copy if provided).

Return type

RegularPentagon

regular_polygon(*points, name=None, copy=None)

Add a regular polygon or return a copy.

Assumptions & constraints: at least three ordered points; all sides and angles are constrained equal; polygon is inscribed in a circle.

Parameters

- ***points** (*Point*) – Vertices in order.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RegularPolygon* | *None*) – Existing regular polygon to reuse.

Returns

The polygon instance (copy if provided).

Return type

RegularPolygon

rhomboid(p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None)

Add a rhomboid or return a copy.

Opposite sides parallel; adjacent sides unequal; no right-angle constraint. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Rhomboid* | *None*) – Existing rhomboid to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Rhomboid

rhombus(p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None)

Add a rhombus or return a copy.

All sides equal; opposite sides parallel; angles not necessarily 90°. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Rhombus* | *None*) – Existing rhombus to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Rhombus

right_isosceles_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add a right isosceles triangle or return a copy.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B (right angle at B).
- **p3** (*Point* | *None*) – Vertex C.
- **name** (*str* | *None*) – Optional name.
- **copy** (*RightIsoscelesTriangle* | *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

RightIsoscelesTriangle

Notes

Sides AB and BC are constrained equal (order matters).

right_trapezoid(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a right trapezoid or return a copy.

Constrains AB to be parallel to CD and angles A, D = 90°. Optional convexity enforcement.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B.
- **p3** (*Point* | *None*) – Vertex C.
- **p4** (*Point* | *None*) – Vertex D.
- **name** (*str* | *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*RightTrapezoid* | *None*) – Existing trapezoid to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

RightTrapezoid

right_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add a right triangle or return a copy.

Parameters

- **p1** (*Point* | *None*) – Vertex A.
- **p2** (*Point* | *None*) – Vertex B (right angle at B).
- **p3** (*Point* | *None*) – Vertex C.

- **name** (*str* / *None*) – Optional name.
- **copy** (*RightTriangle* / *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

RightTriangle

scalene_triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add a scalene triangle or return a copy.

Parameters

- **p1** (*Point* / *None*) – Vertex A.
- **p2** (*Point* / *None*) – Vertex B.
- **p3** (*Point* / *None*) – Vertex C.
- **name** (*str* / *None*) – Optional name.
- **copy** (*ScaleneTriangle* / *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

ScaleneTriangle

Notes

All side lengths are constrained to be distinct.

semicircle(*center, A, B, name=None*)

Add a semicircle (180° major arc) from A to B with a given center.

Enforces collinearity of *center*, *A*, and *B* (direction not forced).

Parameters

- **center** (*Point*) – Center of the circle.
- **A** (*Point*) – First endpoint on the circle.
- **B** (*Point*) – Second endpoint on the circle.
- **name** (*str* / *None*) – Optional name. Defaults to *semicircle_A_B_center_<center>*.

Returns

The created semicircle as a major arc.

Return type

MajorArc

square(*p1=None, p2=None, p3=None, p4=None, name=None, copy=None*)

Add a square or return a copy.

All sides equal, all angles 90°, opposite sides parallel.

Parameters

- **p1** (*Point* / *None*) – Vertex A.
- **p2** (*Point* / *None*) – Vertex B.
- **p3** (*Point* / *None*) – Vertex C.
- **p4** (*Point* / *None*) – Vertex D.
- **name** (*str* / *None*) – Optional name.
- **copy** (*Square* / *None*) – Existing square to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Square

trapezoid(*p1=None, p2=None, p3=None, p4=None, name=None, force_convex=False, copy=None*)

Add a trapezoid or return a copy.

Constrains AB to be parallel to CD. Optional convexity enforcement.

Parameters

- **p1** (*Point* / *None*) – Vertex A.
- **p2** (*Point* / *None*) – Vertex B.
- **p3** (*Point* / *None*) – Vertex C.
- **p4** (*Point* / *None*) – Vertex D.
- **name** (*str* / *None*) – Optional name.
- **force_convex** (*bool*) – If True, enforces convexity and simplicity.
- **copy** (*Trapezoid* / *None*) – Existing trapezoid to reuse.

Returns

The quadrilateral instance (copy if provided).

Return type

Trapezoid

triangle(*p1=None, p2=None, p3=None, name=None, copy=None*)

Add a general triangle or return a copy.

Parameters

- **p1** (*Point* / *None*) – Vertex A.
- **p2** (*Point* / *None*) – Vertex B.
- **p3** (*Point* / *None*) – Vertex C.
- **name** (*str* / *None*) – Optional name.
- **copy** (*Triangle* / *None*) – Existing triangle to reuse.

Returns

The triangle instance (copy if provided).

Return type

Triangle

Notes

Three distinct points are required when creating.

ADD RELATIONSHIPS

RelationshipNamespace class provides geometric relationship constraints between various geometric objects such as points, lines, circles, arcs, and polygons. These constraints can be used within a scene to enforce geometric properties like parallelism, perpendicularity, collinearity, congruence, similarity, and containment.

Each method adds symbolic constraints to the scene's constraint solver to represent the desired geometric relationships.

class pygeox.relationship.**RelationshipNamespace**(*scene*)

Bases: object

Namespace class for defining geometric relationship constraints in a scene.

scene

The geometric scene containing objects and constraint solvers.

acute_angle(*args, **kwargs)

angle_bisector(*args, **kwargs)

collinear(*args, **kwargs)

congruent(*args, **kwargs)

inside(*args, **kwargs)

is_chord(*args, **kwargs)

is_diameter(*args, **kwargs)

is_midpoint(*args, **kwargs)

is_radius(*args, **kwargs)

line_extension_intersects_circle_at(*args, **kwargs)

line_extensions_intersect_at(*args, **kwargs)

line_intersects_circle_at(*args, **kwargs)

lines_intersect_at(*args, **kwargs)

mirror_across_line(*args, **kwargs)

obtuse_angle(*args, **kwargs)

outside(*args, **kwargs)
parallel(*args, **kwargs)
perpendicular(*args, **kwargs)
perpendicular_bisector_at(*args, **kwargs)
point_lies_on(*args, **kwargs)
points_lie_on(*args, **kwargs)
right_angle(*args, **kwargs)
rotation_around_point(*args, **kwargs)
scale(*args, **kwargs)
similar(*args, **kwargs)
tangent_to_circle(*args, **kwargs)
translation(*args, **kwargs)

ADD CONSTRAINTS

```
class pygeox.constraint.ConstraintNamespace(scene)
```

Bases: object

```
eq(a, b, description=None)
```

Adds an equality constraint ($a == b$) to the scene. If ‘a’ is a square root expression, it squares both sides.

Parameters

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type

Equality | *None*

```
geq(a, b, description=None)
```

Adds a ‘greater than or equal to’ constraint ($a \geq b$) to the scene.

Parameters

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type

GreaterThan | *None*

```
gt(a, b, description=None)
```

Adds a ‘greater than’ constraint ($a > b$) to the scene.

Parameters

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type

StrictGreaterThan | *None*

```
leq(a, b, description=None)
```

Adds a ‘less than or equal to’ constraint ($a \leq b$) to the scene.

Parameters

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type

LessThan | *None*

```
lt(a, b, description=None)
```

Adds a ‘less than’ constraint ($a < b$) to the scene.

Parameters

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type*StrictLessThan* | *None***neq**(*a*, *b*, *description=None*)Adds an inequality constraint (*a* != *b*) to the scene.**Parameters**

- **a** (*Expr* | *int* | *float*)
- **b** (*Expr* | *int* | *float*)
- **description** (*str* | *None*)

Return type*Unequality* | *None*

SOLVERS

```
class pygeox.solver.SolverNamespace(scene)
```

Bases: object

analytical()

Attempt to solve the constraint system symbolically (analytically).

Steps:

- Substitute mapping constraints.
- Separate equations and inequalities.
- Solve equalities using SymPy.
- Filter and validate solutions.
- Return unique solution or multiple candidates.

Raises

ValueError – If no solution or invalid solutions found.

Returns

Solution mapping symbols to values.

Return type

dict or list of dicts

```
get_objective_function(variables, equations, alpha=1.0, beta=1.0, strict_penalty=1.0,
                        strict_tol=0.0001, use_numba=False)
```

Build the objective function for numerical optimization from constraints.

Supports both pure Python and Numba-jitted versions.

Parameters

- **variables** – List of variables to optimize.
- **equations** – List of (description, sympy expression) constraints.
- **alpha** (*float*) – Weight for equality constraints.
- **beta** (*float*) – Weight for inequality constraints.
- **strict_penalty** (*float*) – Penalty for strict inequality violations.
- **strict_tol** (*float*) – Tolerance for strict inequalities.
- **use_numba** (*bool*) – If True, return a Numba-accelerated function.

Returns

(objective function, variables, dictionary of sub-objectives)

Return type

tuple

```
has_any_overlapping_points(threshold=0.0001)
```

```
numerical(method='basinhopping', seed=None, alpha=1.0, beta=1.0, strict_penalty=1.0,
           strict_tol=0.0001, distance_penalty=1, min_dist=None, use_numba=True,
           save_param_history=False, verbose=True)
```

Solve the constraint system numerically using global or local optimization.

Supports methods like basinhopping, L-BFGS-B, and dual_annealing.

Parameters

- **method** – Optimization method name.
- **seed** – Random seed.
- **alpha** – Penalty weights and tolerances.
- **beta** – Penalty weights and tolerances.
- **strict_penalty** – Penalty weights and tolerances.
- **strict_tol** – Penalty weights and tolerances.
- **distance_penalty** (*float*) – Weight for penalty avoiding degenerate configurations.
- **min_dist** (*float* / *None*) – Minimum allowed distance between points.
- **use_numba** (*bool*) – Use Numba-accelerated objective function.
- **save_param_history** (*bool*) – Save parameter values during optimization.
- **verbose** (*bool*) – Print detailed solver progress.

Returns

Dictionary with all objects with scene and solved numerical values.

Return type

dict

Raises

ValueError – If optimization fails.

numerical_analysis(*x=None, verbose=True*)

Evaluate and report the quality of a numerical solution.

Prints objective value, satisfaction of equality, inequality, strict, and not-equal constraints, and warns about overlapping points.

Parameters

- **x** – Optional solution vector to evaluate.
- **verbose** – Whether to print detailed report.

Returns

Analysis summary including objective and constraint values.

Return type

dict

pick_solution(*index=None, ind_var_values=None*)

Select and apply one solution among multiple symbolic solutions.

Parameters

- **index** (*int*) – Index of the solution to pick (random if None).
- **ind_var_values** (*dict*) – Optional fixed values for independent variables.

Raises

ValueError – If solution contains invalid values or fails inequalities.

simplify_constraints(*max_linear=0*)

Simplify the constraint system by applying known variable mappings, removing trivial equalities, and optionally performing cheap linear eliminations.

This reduces the number of independent variables and constraints, aiding solver efficiency.

Parameters

max_linear (*int*) – Maximum number of additional cheap linear eliminations to attempt.

Returns

- **indep_vars** (list[*sympy.Symbol*]): Independent variables after simplification.
- **reduced_eqs** (list[*sympy.Expr*]): Simplified constraint expressions (lhs - rhs).
- **mapping** (dict[*sympy.Symbol*, *sympy.Expr*]): Mapping from dependent to independent variables.

- `red_full_mapping` (dict[sympy.Symbol, sympy.Expr]): Final mapping of variables to expressions.

Return type

tuple

static `substitute_in_obj`(*obj*, *sol*, *numeric=False*, *visited=None*)**Parameters****numeric** (*bool*)

BASIC OBJECTS (POINT, LINES, CIRCLES, ARCS)

`basic_objects.py`: Fundamental 2D geometric primitives with symbolic or numeric coordinates.

Defines `Point`, `LineLike`, `LineSegment`, `Line`, `Ray`, `Circle`, and `Arc` classes with methods for geometric computations such as distance, angle, slope, and length.

class `pygeox.basic_objects.Angle`(*A*, *vertex*, *B*, *plot_sign=True*, *plot_text=False*, *name=None*)

Bases: `object`

Represents an angle defined by three points with a specified vertex.

Parameters

- **A** (*Point*)
- **vertex** (*Point*)
- **B** (*Point*)
- **plot_sign** (*bool* | *None*)
- **plot_text** (*bool* | *None*)
- **name** (*str* | *None*)

A

One endpoint of the angle.

Type

`Point`

vertex

The vertex point of the angle.

Type

`Point`

B

The other endpoint of the angle.

Type

`Point`

plot

Whether to plot the angle.

Type

`bool`

name

Name of the angle.

Type
str

Properties:

value: The angle measure at the vertex in degrees (or symbolic expression).

property value

Calculates and returns the angle value at the vertex formed by points A and B.

Returns

The angle measure in degrees (or symbolic expression if symbolic).

Return type

float or sympy.Expr

```
class pygeoX.basic_objects.BaseArc(center, start_point, end_point, name=None,
                                   arc_type='Arc')
```

Bases: object

Base class for arcs, defined by a center and two endpoints. Not intended to be used directly.

Parameters

- **center** (*Point*)
- **start_point** (*Point*)
- **end_point** (*Point*)
- **name** (*str* / *None*)

center

The center of the arc's circle.

Type
Point

start_point

The start point of the arc.

Type
Point

end_point

The end point of the arc.

Type
Point

name

The name of the arc.

Type
str

arc_type

The type of arc ("Arc", "MinorArc", "MajorArc").

Type
str

start_radius_line

Line from center to start_point.

Type
Line

end_radius_line

Line from center to end_point.

Type

Line

property radius

Returns the radius of the arc (distance from center to start_point).

Returns

The radius.

Return type

sympy.Expr

class pygeoX.basic_objects.**Circle**(center, radius, name=None)

Bases: object

Represents a circle defined by its center Point and radius (symbolic or numeric).

Parameters

- **center** (*Point*)
- **radius** (*int | float | Expr | None*)
- **name** (*str | None*)

center

The center of the circle.

Type

Point

radius

The radius (can be symbolic or numeric).

Type

Coordinate

name

The name of the circle.

Type

str

property area

Returns the area of the circle.

Returns

The area.

Return type

sympy.Expr

property diameter

Returns the diameter (length) of the circle.

Returns

The diameter ($2 * \text{radius}$).

Return type

sympy.Expr

property perimeter

Returns the circumference of the circle.

Returns

The circumference.

Return type

sympy.Expr

class pygeo.basic_objects.**Line**(*point1*, *point2*, *name=None*)

Bases: LineLike

Represents an infinite line defined by two Point objects.

Parameters

- **point1** (*Point*)
- **point2** (*Point*)
- **name** (*str* / *None*)

property equation: ExprReturns the equation of the line in the form $Ax + By + C = 0$. The result is a SymPy expression that evaluates to zero for points on the line.**Returns**

The symbolic equation of the line.

Return type

sympy.Expr

class pygeo.basic_objects.**LineLike**(*point1*, *point2*, *name=None*)

Bases: object

Base class for geometric entities that are line-like (LineSegment, Line, Ray). Defined by two Point objects.

Parameters

- **point1** (*Point*)
- **point2** (*Point*)
- **name** (*str* / *None*)

property direction_vector: tuple[int | float | Expr, int | float | Expr]

Returns the direction vector of the ray, from start_point to direction_point.

Returns

The (dx, dy) vector components.

Return type

tuple[sympy.Expr, sympy.Expr]

property intercept: int | float | ExprReturns the y-intercept (c) of the infinite line defined by point1 and point2 (in the form $y = mx + c$).**Returns**

The y-intercept, or sympy.nan for vertical lines.

Return type

sympy.Expr

property slope: int | float | Expr

Returns the slope of the infinite line passing through point1 and point2.

Returns

The slope, or sympy.oo (symbolic infinity) for vertical lines.

Return type

sympy.Expr

```
class pygeox.basic_objects.LineSegment(point1, point2, name=None)
```

Bases: `LineLike`

Represents a line segment defined by two `Point` objects. This class contains methods specific to a finite segment.

Parameters

- **point1** (*Point*)
- **point2** (*Point*)
- **name** (*str* | *None*)

```
angle(other_line_segment, signed=False)
```

Returns the angle between this line segment and another line segment at their common intersection point (vertex).

Parameters

- **other_line_segment** (*LineSegment*) – The other line segment.
- **signed** (*bool*) – If True, returns the signed angle.

Returns

The angle in degrees.

Return type

`sympy.Expr`

Raises

ValueError – If line segments do not meet at a single common vertex.

```
property length: Expr
```

Returns the length of the line segment.

Returns

The distance between point1 and point2.

Return type

`sympy.Expr`

```
property midpoint: Point
```

Returns the midpoint of the line segment.

Returns

The midpoint as a new `Point` object.

Return type

`Point`

```
class pygeox.basic_objects.MajorArc(center, start_point, end_point, name=None)
```

Bases: `BaseArc`

Represents the major arc (larger arc) between two points on a circle.

Parameters

- **center** (*Point*)
- **start_point** (*Point*)
- **end_point** (*Point*)
- **name** (*str* | *None*)

```
property area
```

Returns the area of the sector defined by the major arc.

Returns

The sector area.

Return type

sympy.Expr

property central_angle

Returns the central angle of the major arc in degrees (180 to 360).

Returns

The angle in degrees.

Return type

sympy.Expr

property inscribed_angle

Returns the inscribed angle subtended by the arc in degrees.

Returns

The inscribed angle in degrees.

Return type

sympy.Expr

property length

Returns a non-standard 'length' of the major arc, calculated as radius * (angle in degrees).

Note: Standard arc length requires the angle in radians ($s = r * \text{theta_radians}$).**Returns**

The arc length.

Return type

sympy.Expr

property midpoint: Point

Calculates and returns the midpoint of the major arc.

Parameters

- **arc** (*MajorArc*) – The MajorArc instance.
- **name** (*str*, *optional*) – Name for the midpoint. If None, auto-generated.

Returns

The midpoint as a new Point object.

Return type

Point

mirror_across_line(*axis_line*, *name=None*)**Parameters****axis_line** (*Line*)**Return type***MinorArc***class** pygeoX.basic_objects.**MinorArc**(*center*, *start_point*, *end_point*, *name=None*)

Bases: BaseArc

Represents the minor arc (smaller arc) between two points on a circle.

Parameters

- **center** (*Point*)
- **start_point** (*Point*)
- **end_point** (*Point*)
- **name** (*str* | *None*)

property area

Returns the area of the sector defined by the minor arc.

Returns

The sector area.

Return type

sympy.Expr

property central_angle

Returns the central angle of the minor arc in degrees (0 to 180).

Returns

The angle in degrees.

Return type

sympy.Expr

property inscribed_angle

Returns the inscribed angle subtended by the arc in degrees.

Returns

The inscribed angle in degrees.

Return type

sympy.Expr

property length

Returns a non-standard 'length' of the minor arc, calculated as radius * (angle in degrees).

Note: Standard arc length requires the angle in radians ($s = r * \text{theta_radians}$).

Returns

The arc length.

Return type

sympy.Expr

class pygeox.basic_objects.**Point**(*name*, *x*, *y*)

Bases: object

Represents a 2D point with symbolic or numeric coordinates.

Parameters

- **name** (*str*)
- **x** (*int* | *float* | *Expr* | *None*)
- **y** (*int* | *float* | *Expr* | *None*)

name

Point name.

Type

str

x

X coordinate.

Type

Coordinate

y

Y coordinate.

Type

Coordinate

angle(*point_b*, *point_c*, *signed=False*)

Compute angle at self between vectors to point_b and point_c in degrees.

Parameters

- **point_b** (*Point*) – First point defining angle arm.
- **point_c** (*Point*) – Second point defining angle arm.
- **signed** (*bool*) – If True, return signed angle (CCW positive).

Returns

Angle in degrees.

Return type

`sympy.Expr`

distance(*other_point*)

Compute the Euclidean distance to another point.

Parameters

other_point (*Point*) – The other point.

Returns

The distance between self and *other_point*.

Return type

`sympy.Expr`

distance_squared(*other_point*)

Compute the squared Euclidean distance to another point.

Parameters

other_point (*Point*) – The other point.

Returns

The squared distance between self and *other_point*.

Return type

`sympy.Expr`

class `pygeox.basic_objects.Ray`(*start_point*, *direction_point*, *name=None*)

Bases: `LineLike`

Represents a ray defined by a starting `Point` and a second `Point` indicating its direction. The ray extends infinitely from the *start_point* through the *direction_point*.

Parameters

- **start_point** (*Point*)
- **direction_point** (*Point*)
- **name** (*str* | *None*)

property length: `oo`

Returns the length of the ray, which is infinite.

Returns

Symbolic infinity.

Return type

`sympy.oo`

POLYGONS

custom_objects.py: Classes for polygons and specialized geometric shapes within a geometric scene.

Defines general Polygon and its subclasses including CyclicPolygon, RegularPolygon, and various specific polygons like Triangles, Quadrilaterals, and their special cases (e.g., EquilateralTriangle, Rectangle, Square).

These classes encapsulate geometric properties, constraints, and relationships, enabling symbolic geometry modeling, constraint solving, and scene management.

```
class pygeox.custom_objects.AcuteTriangle(geoscene, p1, p2, p3, name=None,  
                                           add_to_scene_as_type='AcuteTriangle')
```

Bases: Triangle

Triangle with all angles < 90 degrees.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

```
class pygeox.custom_objects.CyclicPolygon(geoscene, *points, name=None,  
                                           add_to_scene_as_type='CyclicPolygon')
```

Bases: Polygon

Polygon whose vertices all lie on a single circle (circumcircle). Circumcenter and circumradius are always defined.

Parameters

- **points** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

property circumcenter

property circumradius

```
class pygeox.custom_objects.EquilateralTriangle(geoscene, p1, p2, p3, name=None,  
                                                add_to_scene_as_type='EquilateralTriangle')
```

Bases: Triangle

Triangle with all sides equal and all angles 60 degrees.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

property circumradius

property inradius

```
class pygeox.custom_objects.IsoscelesTrapezoid(geoscene, p1, p2, p3, p4, name=None,  
                                                add_to_scene_as_type='IsoscelesTrapezoid',  
                                                force_convex=False)
```

Bases: Trapezoid

Trapezoid with non-parallel sides equal in length. IsoscelesTrapezoid(A,B,C,D) means AB is parallel to CD and BC and DA have same length.

Parameters

force_convex (*bool*)

```
class pygeox.custom_objects.IsoscelesTriangle(geoscene, p1, p2, p3, name=None,  
                                              add_to_scene_as_type='IsoscelesTriangle')
```

Bases: Triangle

Triangle with at least two equal sides. IsoscelesTriangle(A,B,C) means that AB=BC.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

```
class pygeox.custom_objects.Kite(geoscene, p1, p2, p3, p4, name=None,  
                                 add_to_scene_as_type='Kite', force_convex=False)
```

Bases: Quadrilateral

Quadrilateral with two distinct pairs of adjacent sides equal.

Parameters

force_convex (*bool*)

property area

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

sympy.Expr

```
class pygeoX.custom_objects.ObtuseTriangle(geoscene, p1, p2, p3, name=None,
                                           add_to_scene_as_type='ObtuseTriangle')
```

Bases: Triangle

Triangle with one angle > 90 degrees.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* | *None*)
- **add_to_scene_as_type** (*str* | *None*)

```
class pygeoX.custom_objects.Parallelogram(geoscene, p1, p2, p3, p4, name=None,
                                           add_to_scene_as_type='Parallelogram',
                                           force_convex=False)
```

Bases: Quadrilateral

Quadrilateral with both pairs of opposite sides parallel.

Parameters**force_convex** (*bool*)**property area**

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

sympy.Expr

```
class pygeoX.custom_objects.Polygon(geoscene, *points, name=None,
                                     add_to_scene_as_type='Polygon', force_convex=False)
```

Bases: object

Represents a general polygon defined by an ordered list of Point objects.

Parameters

- **geoscene** – The geometric scene/context to which this polygon belongs.
- ***points** (*Point*) – The vertices of the polygon, in order.
- **name** (*str*, *optional*) – Name for the polygon. If None, an automatic name is generated.
- **add_to_scene_as_type** (*str*, *optional*) – Type string for scene registration. Default is 'Polygon'.
- **force_convex** (*bool*, *optional*) – If True, enforces convexity and simplicity constraint (forcing also counterclockwise direction).

Returns

A new Polygon object registered in the scene.

Return type

Polygon

property area

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

sympy.Expr

property centroid

Returns the vertex centroid (average of all vertices).

Returns

The centroid as a Point object.

Return type

Point

property circumcircle

Returns the circumcircle of the polygon if circumcenter and circumradius exist.

Returns

The circumcircle.

Return type

Circle

Raises

NotImplementedError – If circumcenter or circumradius is not defined for this polygon type.

property incircle

Returns the incircle of the polygon if incenter and inradius exist.

Returns

The incircle.

Return type

Circle

Raises

NotImplementedError – If incenter or inradius is not defined for this polygon type.

internal_angle_at_point(*vertex_point*)

Returns the internal angle (in degrees) at a given vertex of the polygon, correctly handling convex and concave cases.

Parameters

vertex_point (*Point*) – The vertex at which to compute the internal angle.

Returns

Internal angle in degrees, in range (0, 360)

Return type

sympy expression

property internal_angles

Returns the internal angles at each vertex. Angles [DAB,ABC,BCD,..] for Polygon(A,B,C,D,...).

Returns

The internal angles at each vertex in degrees.

Return type

list[float]

property num_sides

Returns a string representation of the Polygon object.

Returns

String representation.

Return type

str

property perimeter

Calculates the perimeter of the polygon by summing side lengths.

Returns

The perimeter. Units: same as input coordinates.

Return type

sympy.Expr

property sides: List[LineSegment]

Returns the sides of the polygon as LijneSegment objects, in order. Sides [AB, BC, CD, ...] for Polygon(A,B,C,D,...).

Returns

The sides of the polygon.

Return type

list[LijneSegment]

```
class pygeoX.custom_objects.Quadrilateral(geoscene, p1, p2, p3, p4, name=None,
                                           add_to_scene_as_type='Quadrilateral',
                                           force_convex=False)
```

Bases: Polygon

Represents a general quadrilateral defined by four Point objects. Inherits area, perimeter, sides, and internal_angles from Polygon.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **p4** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)
- **force_convex** (*bool*)

property diagonals**property midsegments**

Returns the two midsegments (lines connecting midpoints of opposite sides) of the quadrilateral.

Returns

The two midsegments.

Return type

list[LineSegment]

```
class pygeoX.custom_objects.Rectangle(geoscene, p1, p2, p3, p4, name=None,
                                       add_to_scene_as_type='Rectangle',
                                       force_convex=False)
```

Bases: Parallelogram

Parallelogram with all angles 90 degrees.

Parameters

- **force_convex** (*bool*)

property area

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

`sympy.Expr`

property circumcenter**property circumradius****property diagonals****property perimeter**

Calculates the perimeter of the polygon by summing side lengths.

Returns

The perimeter. Units: same as input coordinates.

Return type

`sympy.Expr`

```
class pygeoX.custom_objects.RegularHeptagon(geoscene, *points, name=None,  
                                              add_to_scene_as_type='RegularHeptagon')
```

Bases: `RegularPolygon`

A specific type of `RegularPolygon` with 7 sides.

Parameters

- **points** (`Point`)
- **name** (`str` / `None`)
- **add_to_scene_as_type** (`str` / `None`)

```
class pygeoX.custom_objects.RegularHexagon(geoscene, *points, name=None,  
                                              add_to_scene_as_type='RegularHexagon')
```

Bases: `RegularPolygon`

A specific type of `RegularPolygon` with 6 sides.

Parameters

- **points** (`Point`)
- **name** (`str` / `None`)
- **add_to_scene_as_type** (`str` / `None`)

```
class pygeoX.custom_objects.RegularOctagon(geoscene, *points, name=None,  
                                              add_to_scene_as_type='RegularOctagon')
```

Bases: `RegularPolygon`

A specific type of `RegularPolygon` with 8 sides.

Parameters

- **points** (`Point`)
- **name** (`str` / `None`)
- **add_to_scene_as_type** (`str` / `None`)

```
class pygeox.custom_objects.RegularPentagon(geoscene, *points, name=None,
                                             add_to_scene_as_type='RegularPentagon')
```

Bases: RegularPolygon

A specific type of RegularPolygon with 5 sides.

Parameters

- **points** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

```
class pygeox.custom_objects.RegularPolygon(geoscene, *points, name=None,
                                             add_to_scene_as_type='RegularPolygon')
```

Bases: Polygon

Represents a regular polygon (all sides and angles equal). It imposes regularity constraints on the provided points. Defines its own reduced parameter set (center, radius, orientation).

Parameters

- **points** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

property area

Calculates the area of the regular polygon using its properties.

property center

Returns the center point of the regular polygon.

property circumcenter

Returns: *Point*: The circumcenter of the regular polygon, which coincides with the center for regular polygons.

property circumradius

Returns the circumradius of the regular polygon.

property diagonal

Longest diagonal of the regular polygon.

property height

Height (distance from one side to the opposite vertex) of the regular polygon.

property incenter

Returns: *Point*: The incenter of the regular polygon, which coincides with the center for regular polygons.

property inradius

Inradius (apothem) of the regular polygon.

property internal_angle

Returns: *float*: The internal angle (in degrees) of the regular polygon.

property orientation

Returns the orientation (rotation) of the regular polygon in radians.

property perimeter

Calculates the perimeter of the regular polygon using its properties.

property side_length

Side length of the regular polygon.

property width

Width (distance between two farthest separated points) of the regular polygon.

```
class pygeoX.custom_objects.Rhomboid(geoscene, p1, p2, p3, p4, name=None,
                                     add_to_scene_as_type='Rhomboid',
                                     force_convex=False)
```

Bases: Parallelogram

Parallelogram with adjacent sides of unequal lengths and angles not right.

Parameters

force_convex (*bool*)

```
class pygeoX.custom_objects.Rhombus(geoscene, p1, p2, p3, p4, name=None,
                                     add_to_scene_as_type='Rhombus', force_convex=False)
```

Bases: Parallelogram

Parallelogram with all sides equal.

Parameters

force_convex (*bool*)

property area

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

sympy.Expr

property incenter**property inradius**

```
class pygeoX.custom_objects.RightIsoscelesTriangle(geoscene, p1, p2, p3, name=None,
                                                    add_to_scene_as_type='RightIsoscelesTriangle')
```

Bases: RightTriangle, IsoscelesTriangle

Triangle with a right angle and two equal sides. RightIsoscelesTriangle(A,B,C) means that AB=BC and B is the right angle.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

property circumcenter**property circumradius**

```
class pygeoX.custom_objects.RightTrapezoid(geoscene, p1, p2, p3, p4, name=None,
                                             add_to_scene_as_type='IsoscelesTrapezoid',
                                             force_convex=False)
```

Bases: Trapezoid

Trapezoid with non-parallel sides equal in length. RightTrapezoid(A,B,C,D) means AB is parallel to CD and angle A and D are right angles.

Parameters

force_convex (*bool*)

```
class pygeoX.custom_objects.RightTriangle(geoscene, p1, p2, p3, name=None,
                                             add_to_scene_as_type='RightTriangle')
```

Bases: Triangle

Triangle with one right angle. RightTriangle(A,B,C) means that B is the right angle.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

```
class pygeoX.custom_objects.ScaleneTriangle(geoscene, p1, p2, p3, name=None,
                                             add_to_scene_as_type='ScaleneTriangle')
```

Bases: Triangle

Triangle with all sides of different lengths.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

```
class pygeoX.custom_objects.Square(geoscene, p1, p2, p3, p4, name=None,
                                    add_to_scene_as_type='Square', force_convex=False)
```

Bases: Rectangle

Rectangle with all sides equal.

Parameters

force_convex (*bool*)

property area

Calculates the area of the polygon using the shoelace formula.

Returns

The area. Units: square of input coordinates.

Return type

sympy.Expr

property circumcenter

property circumradius

property incenter

property inradius

property perimeter

Calculates the perimeter of the polygon by summing side lengths.

Returns

The perimeter. Units: same as input coordinates.

Return type

sympy.Expr

```
class pygeoX.custom_objects.Trapezoid(geoscene, p1, p2, p3, p4, name=None,  
                                       add_to_scene_as_type='Trapezoid',  
                                       force_convex=False)
```

Bases: Quadrilateral

Quadrilateral with at least one pair of parallel sides. Trapezoid(A,B,C,D) means AB is parallel to CD.

Parameters

force_convex (*bool*)

```
class pygeoX.custom_objects.Triangle(geoscene, p1, p2, p3, name=None,  
                                       add_to_scene_as_type='Triangle')
```

Bases: Polygon

Represents a general triangle defined by three Point objects.

Parameters

- **p1** (*Point*)
- **p2** (*Point*)
- **p3** (*Point*)
- **name** (*str* / *None*)
- **add_to_scene_as_type** (*str* / *None*)

property altitudes: *list*

Returns: list[LineSegment]: The three altitudes of the triangle, each from a vertex perpendicular to the opposite side.

property angle_bisectors: *list*

Returns: list[LineSegment]: The three angle bisectors of the triangle, each bisecting one angle and ending at the opposite side.

property area

Calculates the area of the triangle using Heron's formula.

property circumcenter

property circumradius

property heights

Returns the heights from each vertex to the opposite side.

property incenter

property inradius

property medians: list

Returns: list[LineSegment]: The three medians of the triangle, each from a vertex to the mid-point of the opposite side.

property midsegments: list

Returns: list[LineSegment]: The three midsegments of the triangle, each connecting the mid-points of two sides.

property orthocenter

ANIMATION (UNDER CONSTRUCTION)

class pygeox.animate.**AnimateNamespace**(*scene*)

Bases: object

Namespace to manage animation parameters and generate animations by varying parameters within a geometric scene.

Provides functionality to add animation parameters, run animations by solving constraints at discrete parameter values, and export animations as GIFs.

add_parameter(*name*, *total_time=1*)

Add a new symbolic animation parameter to the scene.

Parameters

- **name** (*str*) – The name of the animation parameter.
- **total_time** (*int* | *float* | *Expr* | *Symbol*)

Returns

The sympy.Symbol representing the animation parameter.

eval_piecewise_at_zero(*expr*, *anim_syms*)

If *expr* contains Piecewise, pick the first branch whose condition is: - literally True, or - an And(..., alpha >= 0, ...) (ignoring other bounds), or - becomes True after substituting alpha -> 0. Otherwise (no Piecewise), return *expr* unchanged.

get_time_params()

Retrieve all animation parameters currently registered in the scene.

Returns

List of sympy.Symbol animation parameters.

gif(*save_loc='animation.gif'*)

Generate and save an animated GIF from the current list of solved scenes.

Parameters

- **save_loc** (*str*) – File path to save the GIF.
- **total_time** – Total duration of the GIF in seconds.

Raises

ValueError – If there are no frames to generate.

Return type

None

point_sweep_arc(*point*, *arc*, *counterclockwise=True*, *total_time=2*, *speed=None*)

Animate a point moving along an arc by directly constraining its x and y coordinates. Point moves from *arc.start_point* to *arc.end_point* over the arc length.

Parameters

- **point** (*Point*)
- **arc** (*MinorArc* | *MajorArc*)

- **counterclockwise** (*bool*)
- **total_time** (*int | float | Expr | Symbol | None*)
- **speed** (*int | float | Expr | Symbol | None*)

point_sweep_circle(*point, circle, counterclockwise=True, total_time=2, speed=None*)

Animate a point moving around a circle by directly constraining its x and y coordinates. Point starts at the top of the circle and moves counterclockwise by default.

Parameters

- **point** (*Point*)
- **circle** (*Circle*)
- **counterclockwise** (*bool*)
- **total_time** (*int | float | Expr | Symbol | None*)
- **speed** (*int | float | Expr | Symbol | None*)

point_sweep_line(*point, start_point, end_point, total_time=2, speed=None*)

Animate a point moving along a line segment by directly setting its x and y coordinates parametrically.

Parameters

- **point** (*Point*)
- **start_point** (*Point*)
- **end_point** (*Point*)
- **total_time** (*int | float | Expr | Symbol | None*)
- **speed** (*int | float | Expr | Symbol | None*)

point_sweep_polyline(*point, path_points, total_time=5, speed=None*)

Animates a point moving smoothly along a polyline defined by a sequence of points.

A single parameter *time_param* in [0, 1] controls the point's position. As *time_param* increases from 0 to 1, the point traverses each segment of the polyline in proportion to segment length. A single sympy.Piecewise constraint ensures that the point's coordinates match the appropriate interpolated position along the active segment.

Parameters

- **point** (*Point*) – The Point object to animate.
- **path_points** (*List[Point]*) – Ordered list of points defining the polyline path.
- **total_time** (*int | float | Expr | Symbol | None*)
- **speed** (*List[int | float | Expr | Symbol] | None*)

run(*moving_points, frames=20, opt_method='dual_annealing', distance_penalty=1, min_dist=None, run_type='independent'*)

Generate a sequence of solved scenes by interpolating animation parameters from 0 to 1, effectively creating frames of the animation.

Points not listed in *moving_points* are fixed to their initial positions.

Parameters

- **moving_points** (*List[Point]*) – A list of Point objects that are allowed to move during animation. Points not in this list will remain fixed.
- **frames** (*int*) – The number of frames to generate in the animation.
- **opt_method** (*str*) – The numerical solver method to use for constraint solving. Refer to *scipy.optimize.minimize* for available options.
- **distance_penalty** (*float | None*) – The penalty weight to avoid degenerate cases related to area and point distances. This is used in the first frame and as a penalty constraint. It is removed from the moving points.
- **min_dist** (*float | None*) – The minimum allowed distance between points. This is only used for the first frame.
- **run_type** (*Literal['independent', 'together']*)

Raises

ValueError – If no animation parameters are defined in the scene.

PYTHON MODULE INDEX

p

`pygeox.constraint`, [35](#)

INDEX

\spxentryA\spxextrapygeox.basic_objects.Angle attribute, 41
 \spxentryacute_angle()\spxextrapygeox.relationship.RelationshipNamespaceproperty, 45
 method, 33
 \spxentryacute_triangle()\spxextrapygeox.add_objects.AddNamespaceproperty, 46
 method, 21
 \spxentryAcuteTriangle\spxextraclass in py-geox.custom_objects, 49
 \spxentryadd_constraint()\spxextrapygeox.geoscene.GeoScene property, 51
 method, 17
 \spxentryadd_mapping_constraint()\spxextrapygeox.geoscene.GeoSceneproperty, 52
 method, 17
 \spxentryadd_object()\spxextrapygeox.geoscene.GeoScene property, 53
 method, 18
 \spxentryadd_parameter()\spxextrapygeox.animate.AnimateNamespaceproperty, 55
 method, 61
 \spxentryadd_parameter()\spxextrapygeox.geoscene.GeoScene property, 56
 method, 18
 \spxentryAddNamespace\spxextraclass in py-geox.add_objects, 21
 \spxentryaltitudes\spxextrapygeox.custom_objects.Triangle property, 58
 \spxentryanalytical()\spxextrapygeox.solver.SolverNamespaceproperty, 37
 method, 37
 \spxentryAngle\spxextraclass in pygeox.basic_objects, 41
 \spxentryangle()\spxextrapygeox.add_objects.AddNamespaceproperty, 42
 method, 21
 \spxentryangle()\spxextrapygeox.basic_objects.LineSegmentproperty, 45
 method, 45
 \spxentryangle()\spxextrapygeox.basic_objects.Point property, 47
 method, 47
 \spxentryangle()\spxextrapygeox.geoscene.GeoScene property, 18
 method, 18
 \spxentryangle_bisector()\spxextrapygeox.relationship.RelationshipNamespaceproperty, 46
 method, 33
 \spxentryangle_bisectors\spxextrapygeox.custom_objects.Triangle property, 58
 property, 58
 \spxentryAnimateNamespace\spxextraclass in py-geox.animate, 61
 \spxentryarc_type\spxextrapygeox.basic_objects.BaseArc attribute, 42
 \spxentryarea\spxextrapygeox.basic_objects.Circle property, 43
 \spxentryarea\spxextrapygeox.basic_objects.MajorArc property, 45
 \spxentryarea\spxextrapygeox.basic_objects.MinorArc property, 47
 \spxentryarea\spxextrapygeox.custom_objects.Kite property, 50
 \spxentryarea\spxextrapygeox.custom_objects.Parallelogram property, 51
 \spxentryarea\spxextrapygeox.custom_objects.Polygon property, 52
 \spxentryarea\spxextrapygeox.custom_objects.Rectangle property, 53
 \spxentryarea\spxextrapygeox.custom_objects.RegularPolygon property, 55
 \spxentryarea\spxextrapygeox.custom_objects.Rhombus property, 56
 \spxentryarea\spxextrapygeox.custom_objects.Square property, 57
 \spxentryarea\spxextrapygeox.custom_objects.Triangle property, 58
 \spxentryB\spxextrapygeox.basic_objects.Angle attribute, 41
 \spxentryBaseArc\spxextraclass in pygeox.basic_objects, 42
 \spxentrycenter\spxextrapygeox.basic_objects.BaseArc attribute, 42
 \spxentrycenter\spxextrapygeox.basic_objects.Circle attribute, 43
 \spxentrycenter\spxextrapygeox.custom_objects.RegularPolygon property, 55
 \spxentrycentral_angle\spxextrapygeox.basic_objects.MajorArc property, 46
 \spxentrycentral_angle\spxextrapygeox.basic_objects.MinorArc property, 47
 \spxentrycentroid\spxextrapygeox.custom_objects.Polygon property, 52
 \spxentrychord()\spxextrapygeox.add_objects.AddNamespaceproperty, 22
 method, 22
 \spxentryCircle\spxextraclass in pygeox.basic_objects, 43

\spxentrycircle()\spxextrapygeox.add_objects.AddNamespace	\spxentryend_point\spxextrapygeox.basic_objects.BaseArc
method, 22	attribute, 42
\spxentrycircumcenter\spxextrapygeox.custom_objects.CyclicPolygon	\spxentryend_radius_line\spxextrapygeox.basic_objects.BaseArc
property, 49	attribute, 42
\spxentrycircumcenter\spxextrapygeox.custom_objects.Rectangle	\spxentryeq()\spxextrapygeox.constraint.ConstraintNamespace
property, 54	method, 35
\spxentrycircumcenter\spxextrapygeox.custom_objects.RegularPolygon	\spxentryequation\spxextrapygeox.basic_objects.Line
property, 55	property, 44
\spxentrycircumcenter\spxextrapygeox.custom_objects.RightIsoscelesTriangle	\spxentryEuclideanTriangle()\spxextrapygeox.add_objects.AddNamespace
property, 56	method, 22
\spxentrycircumcenter\spxextrapygeox.custom_objects.Square	\spxentryEquilateralTriangle\spxextra
property, 57	class in py-
\spxentrycircumcenter\spxextrapygeox.custom_objects.Triangle	geox.custom_objects, 49
property, 58	\spxentryeval_pieewise_at_zero()\spxextrapygeox.animate.AnimateNamespace
\spxentrycircumcircle\spxextrapygeox.custom_objects.Polygon	method, 61
property, 52	\spxentryGeoScene\spxextra
\spxentrycircumradius\spxextrapygeox.custom_objects.CyclicPolygon	class in pygeox.geoscene, 17
property, 49	\spxentryGeoScene.ProvingContext\spxextra
\spxentrycircumradius\spxextrapygeox.custom_objects.EquilateralTriangle	class in py-
property, 50	geox.geoscene, 17
\spxentrycircumradius\spxextrapygeox.custom_objects.Rectangle	\spxentryget()\spxextrapygeox.constraint.ConstraintNamespace
property, 54	method, 35
\spxentrycircumradius\spxextrapygeox.custom_objects.RegularPolygon	\spxentryget_all_objects()\spxextrapygeox.geoscene.GeoScene
property, 55	method, 18
\spxentrycircumradius\spxextrapygeox.custom_objects.RightIsoscelesTriangle	\spxentryget_object()\spxextrapygeox.geoscene.GeoScene
property, 56	method, 18
\spxentrycircumradius\spxextrapygeox.custom_objects.Square	\spxentryget_objective_function()\spxextrapygeox.solver.SolverNamespace
property, 58	method, 37
\spxentrycircumradius\spxextrapygeox.custom_objects.Triangle	\spxentryget_points()\spxextrapygeox.geoscene.GeoScene
property, 58	method, 19
\spxentrycollinear()\spxextrapygeox.relationship.RelationshipNamespace	\spxentryget_time_params()\spxextrapygeox.animate.AnimateNamespace
method, 33	method, 61
\spxentrycongruent()\spxextrapygeox.relationship.RelationshipNamespace	\spxentryget_time_params()\spxextrapygeox.animate.AnimateNamespace
method, 33	method, 61
\spxentryConstraintNamespace\spxextra	\spxentryget()\spxextrapygeox.constraint.ConstraintNamespace
class in py-	method, 35
geox.constraint, 35	\spxentryhas_any_overlapping_points()\spxextrapygeox.solver.SolverNames
\spxentryCyclicPolygon\spxextra	method, 37
class in py-	\spxentryheight\spxextrapygeox.custom_objects.RegularPolygon
geox.custom_objects, 49	property, 55
\spxentrydiagonal\spxextrapygeox.custom_objects.RegularPolygon	\spxentryheights\spxextrapygeox.custom_objects.Triangle
property, 55	property, 58
\spxentrydiagonals\spxextrapygeox.custom_objects.Quadrilateral	\spxentryincenter\spxextrapygeox.custom_objects.RegularPolygon
property, 53	property, 55
\spxentrydiagonals\spxextrapygeox.custom_objects.Rectangle	\spxentryincenter\spxextrapygeox.custom_objects.Rhombus
property, 54	property, 56
\spxentrydiameter\spxextrapygeox.basic_objects.Circle	\spxentryincenter\spxextrapygeox.custom_objects.Square
property, 43	property, 58
\spxentrydirection_vector\spxextrapygeox.basic_objects.LineLike	\spxentryincenter\spxextrapygeox.custom_objects.Triangle
property, 44	property, 58
\spxentrydistance()\spxextrapygeox.basic_objects.Point	\spxentryincircle\spxextrapygeox.custom_objects.Polygon
method, 48	property, 52
\spxentrydistance_squared()\spxextrapygeox.basic_objects.Point	\spxentryinradius\spxextrapygeox.custom_objects.EquilateralTriangle
method, 48	property, 50

<code>\spxentryinradius\spxextrapygeox.custom_objects.RegularPolygon</code>	<code>\spxentryline()\spxextrapygeox.add_objects.AddNamespace</code>
property, 55	method, 23
<code>\spxentryinradius\spxextrapygeox.custom_objects.Rhombus</code>	<code>\spxentryline_extension_intersects_circle_at()\spxextrapygeox.relationship.</code>
property, 56	method, 33
<code>\spxentryinradius\spxextrapygeox.custom_objects.Square</code>	<code>\spxentryline_extensions_intersect_at()\spxextrapygeox.relationship.Relation</code>
property, 58	method, 33
<code>\spxentryinradius\spxextrapygeox.custom_objects.Triangle</code>	<code>\spxentryline_intersects_circle_at()\spxextrapygeox.relationship.Relationshi</code>
property, 59	method, 33
<code>\spxentryinscribed_angle\spxextrapygeox.basic_objects.MajorArc</code>	<code>\spxentryline_segment()\spxextrapygeox.add_objects.AddNamespace</code>
property, 46	method, 24
<code>\spxentryinscribed_angle\spxextrapygeox.basic_objects.MinorArc</code>	<code>\spxentryLineLike\spxextraclass in pygeox.basic_objects,</code>
property, 47	44
<code>\spxentryinside()\spxextrapygeox.relationship.RelationshipNamespace</code>	<code>\spxentrylines_intersect_at()\spxextrapygeox.relationship.RelationshipName</code>
method, 33	method, 33
<code>\spxentryintercept\spxextrapygeox.basic_objects.LineLike</code>	<code>\spxentryLineSegment\spxextraclass in py-</code>
property, 44	geox.basic_objects, 44
<code>\spxentryinternal_angle\spxextrapygeox.custom_objects.RegularPolygon</code>	<code>\spxentrymodule\spxextrapygeox.constraint.ConstraintNamespace</code>
property, 55	method, 35
<code>\spxentryinternal_angle_at_point()\spxextrapygeox.custom_objects.Polygon</code>	<code>\spxentrymajor_arc()\spxextrapygeox.add_objects.AddNamespace</code>
method, 52	method, 24
<code>\spxentryinternal_angles\spxextrapygeox.custom_objects.Polygon</code>	<code>\spxentryMajorArc\spxextraclass in pygeox.basic_objects,</code>
property, 52	45
<code>\spxentryis_chord()\spxextrapygeox.relationship.RelationshipNamespace</code>	<code>\spxentrymedians\spxextrapygeox.custom_objects.Triangle</code>
method, 33	property, 59
<code>\spxentryis_diameter()\spxextrapygeox.relationship.RelationshipNamespace</code>	<code>\spxentrymidpoint\spxextrapygeox.basic_objects.LineSegment</code>
method, 33	property, 45
<code>\spxentryis_midpoint()\spxextrapygeox.relationship.RelationshipNamespace</code>	<code>\spxentrymidpoint\spxextrapygeox.basic_objects.MajorArc</code>
method, 33	property, 46
<code>\spxentryis_radius()\spxextrapygeox.relationship.RelationshipNamespace</code>	<code>\spxentrymidsegments\spxextrapygeox.custom_objects.Quadrilateral</code>
method, 33	property, 53
<code>\spxentryisosceles_trapezoid()\spxextrapygeox.add_objects.AddNamespace</code>	<code>\spxentrymidsegments\spxextrapygeox.custom_objects.Triangle</code>
method, 23	property, 59
<code>\spxentryisosceles_triangle()\spxextrapygeox.add_objects.AddNamespace</code>	<code>\spxentryminor_arc()\spxextrapygeox.add_objects.AddNamespace</code>
method, 23	method, 24
<code>\spxentryIsoscelesTrapezoid\spxextraclass in py-</code>	<code>\spxentryMinorArc\spxextraclass in py-</code>
geox.custom_objects, 50	geox.basic_objects, 46
<code>\spxentryIsoscelesTriangle\spxextraclass in py-</code>	<code>\spxentrymirror_across_line()\spxextrapygeox.basic_objects.MajorArc</code>
geox.custom_objects, 50	method, 46
<code>\spxentryKite\spxextraclass in pygeox.custom_objects, 50</code>	<code>\spxentrymirror_across_line()\spxextrapygeox.relationship.RelationshipName</code>
<code>\spxentrykite()\spxextrapygeox.add_objects.AddNamespace</code>	method, 33
method, 23	<code>\spxentrymodule</code>
<code>\spxentrylength\spxextrapygeox.basic_objects.LineSegment</code>	<code>\spxentrypygeox.add_objects, 21</code>
property, 45	<code>\spxentrypygeox.animate, 61</code>
<code>\spxentrylength\spxextrapygeox.basic_objects.MajorArc</code>	<code>\spxentrypygeox.basic_objects, 41</code>
property, 46	<code>\spxentrypygeox.constraint, 35</code>
<code>\spxentrylength\spxextrapygeox.basic_objects.MinorArc</code>	<code>\spxentrypygeox.custom_objects, 49</code>
property, 47	<code>\spxentrypygeox.geoscene, 17</code>
<code>\spxentrylength\spxextrapygeox.basic_objects.Ray</code>	<code>\spxentrypygeox.relationship, 33</code>
property, 48	<code>\spxentrypygeox.solver, 37</code>
<code>\spxentryleq()\spxextrapygeox.constraint.ConstraintNamespace</code>	<code>\spxentryname\spxextrapygeox.basic_objects.Angle</code>
method, 35	attribute, 41
<code>\spxentryLine\spxextraclass in pygeox.basic_objects, 44</code>	<code>\spxentryname\spxextrapygeox.basic_objects.BaseArc</code>
	attribute, 42

\spxentryname\spxextrapygeox.basic_objects.Circle attribute, 43	\spxentrypoint_lies_on()\spxextrapygeox.relationship.RelationshipNamespaces method, 34
\spxentryname\spxextrapygeox.basic_objects.Point attribute, 47	\spxentrypoint_sweep_arc()\spxextrapygeox.animate.AnimateNamespaces method, 61
\spxentryneq()\spxextrapygeox.constraint.ConstraintNamespaces method, 36	\spxentrypoint_sweep_circle()\spxextrapygeox.animate.AnimateNamespaces method, 62
\spxentrynum_sides\spxextrapygeox.custom_objects.Polygon property, 52	\spxentrypoint_sweep_line()\spxextrapygeox.animate.AnimateNamespaces method, 62
\spxentrynumerical()\spxextrapygeox.solver.SolverNamespaces method, 37	\spxentrypoint_sweep_polyline()\spxextrapygeox.animate.AnimateNamespaces method, 62
\spxentrynumerical_analysis()\spxextrapygeox.solver.SolverNamespaces method, 38	\spxentrypoints()\spxextrapygeox.add_objects.AddNamespaces method, 26
\spxentryobtuse_angle()\spxextrapygeox.relationship.RelationshipNamespaces method, 33	\spxentrypoints_lie_on()\spxextrapygeox.relationship.RelationshipNamespaces method, 34
\spxentryobtuse_triangle()\spxextrapygeox.add_objects.AddNamespaces method, 25	\spxentryPolygon\spxextraclass in py- geox.custom_objects, 51
\spxentryObtuseTriangle\spxextraclass in py- geox.custom_objects, 51	\spxentrypolygon()\spxextrapygeox.add_objects.AddNamespaces method, 26
\spxentryorientation\spxextrapygeox.custom_objects.RegularPolygon property, 55	\spxentryprove()\spxextrapygeox.geoscene.GeoScene method, 19
\spxentryorthocenter\spxextrapygeox.custom_objects.Triangle property, 59	\spxentryproving()\spxextrapygeox.geoscene.GeoScene method, 19
\spxentryoutside()\spxextrapygeox.relationship.RelationshipNamespaces method, 33	\spxentrypygeox.add_objects \spxentrymodule, 21
\spxentryparallel()\spxextrapygeox.relationship.RelationshipNamespaces method, 34	\spxentrypygeox.animate \spxentrymodule, 61
\spxentryParallelogram\spxextraclass in py- geox.custom_objects, 51	\spxentrypygeox.basic_objects \spxentrymodule, 41
\spxentryparallelogram()\spxextrapygeox.add_objects.AddNamespaces method, 25	\spxentrypygeox.constraint \spxentrymodule, 35
\spxentryperimeter\spxextrapygeox.basic_objects.Circle property, 43	\spxentrypygeox.custom_objects \spxentrymodule, 49
\spxentryperimeter\spxextrapygeox.custom_objects.Polygon property, 53	\spxentrypygeox.geoscene \spxentrymodule, 17
\spxentryperimeter\spxextrapygeox.custom_objects.Rectangle property, 54	\spxentrypygeox.relationship \spxentrymodule, 33
\spxentryperimeter\spxextrapygeox.custom_objects.RegularPolygon property, 55	\spxentrypygeox.solver \spxentrymodule, 37
\spxentryperimeter\spxextrapygeox.custom_objects.Square property, 58	\spxentryQuadrilateral\spxextraclass in py- geox.custom_objects, 53
\spxentryperpendicular()\spxextrapygeox.relationship.RelationshipNamespaces method, 34	\spxentryquadrilateral()\spxextrapygeox.add_objects.AddNamespaces method, 26
\spxentryperpendicular_bisector_at()\spxextrapygeox.relationship.RelationshipNamespaces method, 34	\spxentryradius\spxextrapygeox.basic_objects.BaseArc property, 43
\spxentrypick_solution()\spxextrapygeox.solver.SolverNamespaces method, 38	\spxentryradius\spxextrapygeox.basic_objects.Circle attribute, 43
\spxentryplot\spxextrapygeox.basic_objects.Angle tribute, 41	\spxentryRay\spxextraclass in pygeox.basic_objects, 48
\spxentryplot()\spxextrapygeox.geoscene.GeoScene method, 19	\spxentryray()\spxextrapygeox.add_objects.AddNamespaces method, 26
\spxentryPoint\spxextraclass in pygeox.basic_objects, 47	\spxentryRectangle\spxextraclass in py- geox.custom_objects, 53
\spxentrypoint()\spxextrapygeox.add_objects.AddNamespaces method, 25	\spxentryrectangle()\spxextrapygeox.add_objects.AddNamespaces method, 27

\spxentryregular_heptagon()\spxextrapygeox.add_objects.AddNamespace	\spxentryscaleneTriangle\spxextra	class	in	py-geox.custom_objects, 57
method, 27				
\spxentryregular_hexagon()\spxextrapygeox.add_objects.AddNamespace	\spxentrysquare\spxextrapygeox.relationship.RelationshipNamespace			attribute, 33
method, 27				
\spxentryregular_octagon()\spxextrapygeox.add_objects.AddNamespace	\spxentrysquare\spxextrapygeox.add_objects.AddNamespace			method, 30
method, 27				
\spxentryregular_pentagon()\spxextrapygeox.add_objects.AddNamespace	\spxentrysquare_length\spxextrapygeox.custom_objects.RegularPolygon			property, 56
method, 28				
\spxentryregular_polygon()\spxextrapygeox.add_objects.AddNamespace	\spxentrysquare\spxextrapygeox.custom_objects.Polygon			property, 53
method, 28				
\spxentryRegularHeptagon\spxextra	\spxentrysimilar()\spxextrapygeox.relationship.RelationshipNamespace	class	in	py-geox.custom_objects, 54
method, 27				
\spxentryRegularHexagon\spxextra	\spxentrysimplify_constraints()\spxextrapygeox.solver.SolverNamespace	class	in	py-geox.custom_objects, 54
method, 27				
\spxentryRegularOctagon\spxextra	\spxentryslope\spxextrapygeox.basic_objects.LineLike	class	in	py-geox.custom_objects, 54
method, 27				
\spxentryRegularPentagon\spxextra	\spxentrySolverNamespace\spxextra	class	in	py-geox.custom_objects, 54
method, 27				
\spxentryRegularPolygon\spxextra	\spxentrySquare\spxextra	class	in	py-geox.custom_objects, 55
method, 27				
\spxentryRelationshipNamespace\spxextra	\spxentrysquare()\spxextrapygeox.add_objects.AddNamespace	class	in	py-geox.relationship, 33
method, 27				
\spxentryRhomboid\spxextra	\spxentrystart_point\spxextrapygeox.basic_objects.BaseArc	class	in	py-geox.custom_objects, 56
method, 27				
\spxentryrhomboid()\spxextrapygeox.add_objects.AddNamespace	\spxentrystart_radius_line\spxextrapygeox.basic_objects.BaseArc			attribute, 42
method, 28				
\spxentryRhombus\spxextra	\spxentrysubstitute_in_obj()\spxextrapygeox.solver.SolverNamespace	class	in	py-geox.custom_objects, 56
method, 27				
\spxentryrhombus()\spxextrapygeox.add_objects.AddNamespace				static method, 39
method, 28				
\spxentryright_angle()\spxextrapygeox.relationship.RelationshipNamespace	\spxentrytangent_to_circle()\spxextrapygeox.relationship.RelationshipName			method, 34
method, 34				
\spxentryright_isosceles_triangle()\spxextrapygeox.add_objects.AddNamespace	\spxentrytranslation()\spxextrapygeox.relationship.RelationshipNamespace			method, 34
method, 29				
\spxentryright_trapezoid()\spxextrapygeox.add_objects.AddNamespace	\spxentryTrapezoid\spxextra	class	in	py-geox.custom_objects, 58
method, 29				
\spxentryright_triangle()\spxextrapygeox.add_objects.AddNamespace	\spxentrytrapezoid()\spxextrapygeox.add_objects.AddNamespace			method, 30
method, 29				
\spxentryRightIsoscelesTriangle\spxextra	\spxentryTriangle\spxextra	class	in	py-geox.custom_objects, 56
method, 27				
\spxentryRightTrapezoid\spxextra	\spxentrytriangle()\spxextrapygeox.add_objects.AddNamespace	class	in	py-geox.custom_objects, 56
method, 27				
\spxentryRightTriangle\spxextra	\spxentryvalue\spxextrapygeox.basic_objects.Angle	class	in	py-geox.custom_objects, 57
method, 27				
\spxentryrotation_around_point()\spxextrapygeox.relationship.RelationshipNamespace	\spxentryvertex\spxextrapygeox.basic_objects.Angle			property, 42
method, 34				
\spxentryrun()\spxextrapygeox.animate.AnimateNamespace	\spxentrywidth\spxextrapygeox.custom_objects.RegularPolygon			attribute, 41
method, 62				
\spxentryscale()\spxextrapygeox.relationship.RelationshipNamespace				property, 56
method, 34				
\spxentryscalene_triangle()\spxextrapygeox.add_objects.AddNamespace	\spxentryx\spxextrapygeox.basic_objects.Point			attribute, 47
method, 30				
	\spxentryy\spxextrapygeox.basic_objects.Point			attribute, 47